

Data Transfer Instructions

Computer Engineering 1

Motivation

- How do I "move" data?



Type	Distribution
Data transfers	43%
Control flow	23%
Arithmetic	15%
Compare	13%
Logic	5%
Others	1%

- CPU → CPU register to register
- memory/IO → CPU load from memory/IO
- CPU → memory/IO store to memory/IO

Agenda

- **Data Transfers**
- **Register to Register**
- **Loading Literals**
- **Loading Data**
- **Storing Data**
- **Loading/Storing Multiple Registers**
- **The C Perspective**
 - Arrays
 - Pointers

For information on individual instructions covered in these slides:
See also Quick Reference Card for Thumb 16-bit Instruction Set

Learning Objectives

At the end of this lesson you will be able

- to enumerate the 4 transfer types of the Cortex-M0
- to read a Cortex-M0 assembly program with data transfer instructions
- to write assembly programs with the major Cortex-M0 data transfer instructions
- to encode and decode data transfer instructions to/from binary machine code
- to apply the EQU assembler directive
- to explain the concept of a 'literal pool' and to apply the '`LDR Rd,=literal`' pseudo instruction
- to understand pc relative and indirect addressing (including offsets)
- to explain how arrays are stored in memory and how array elements can be accessed
- to understand how a compiler translates array accesses in a C-program to assembly
- to explain how a C-compiler implements pointers and address operators in assembly language

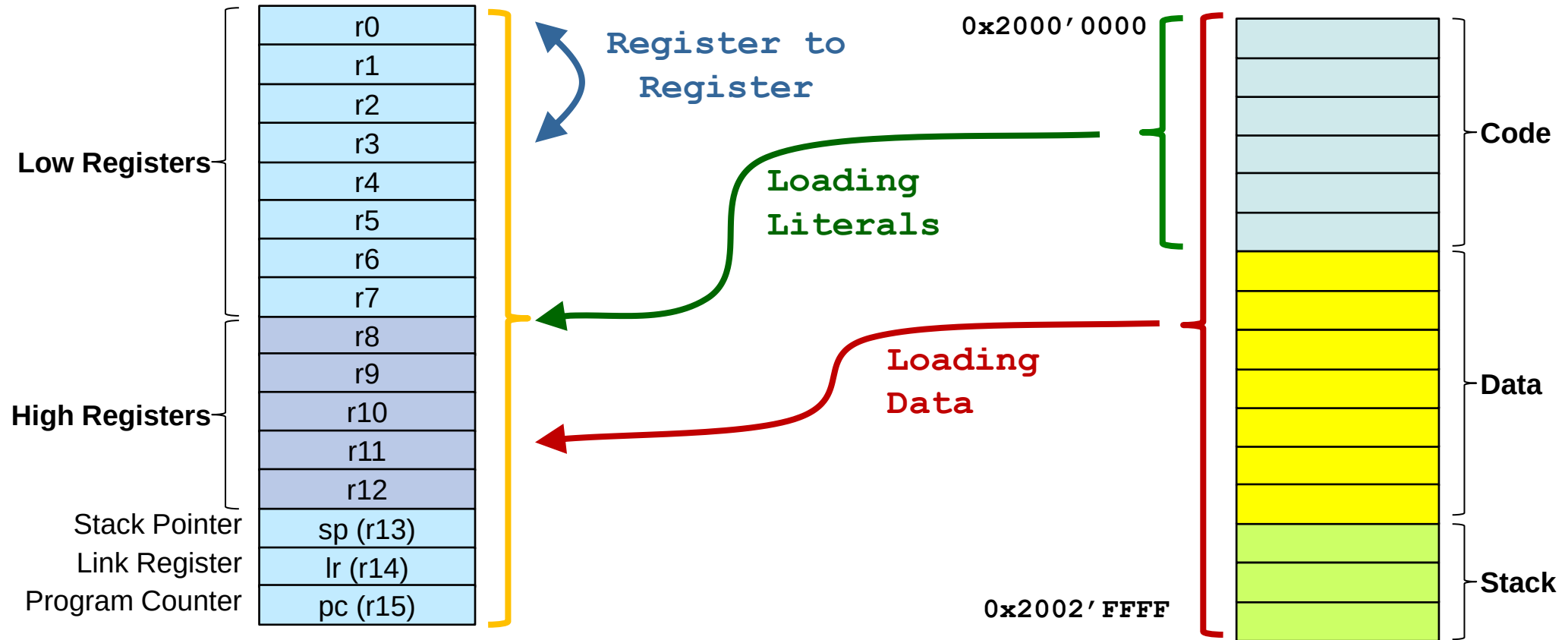
Data Transfers

Architectures

- **Load/Store Architecture (ARM Cortex-M)**
 - Memory accessed only with load/store operations
 - Usual steps for data processing
 - **Load** operands from memory to register
 - Execute operation → result in register
 - **Store** result from register to memory
- **Register Memory Architecture e.g. Intel x86**
 - One of the operands can be located in memory
 - Result can be directly written to memory

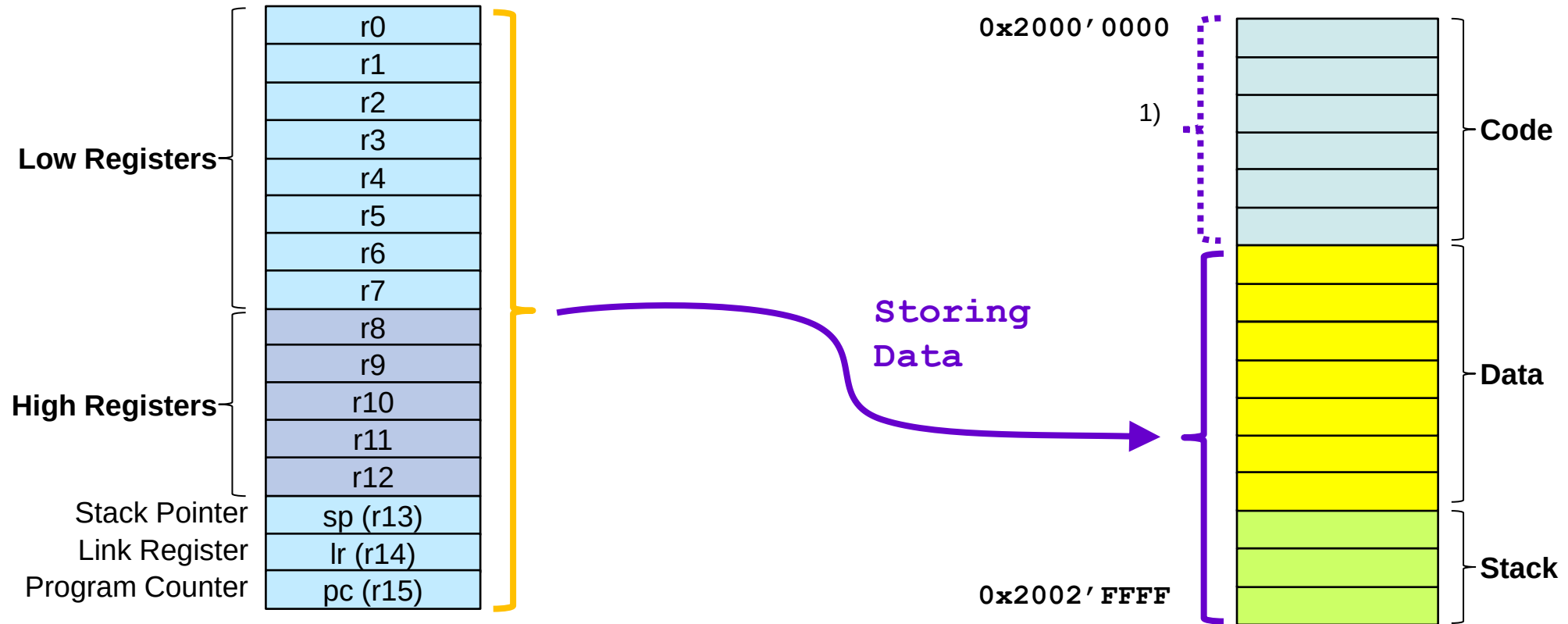
Data Transfers

Transfer Types



Data Transfers

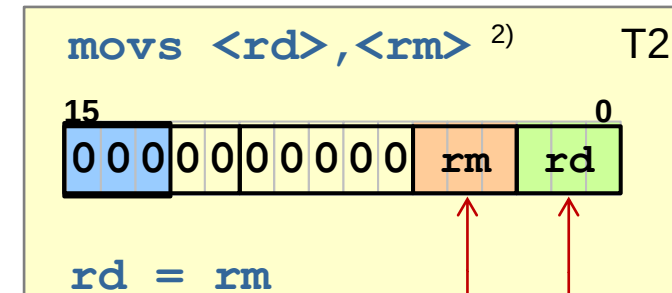
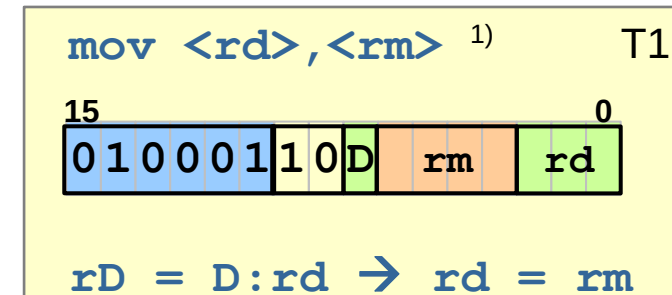
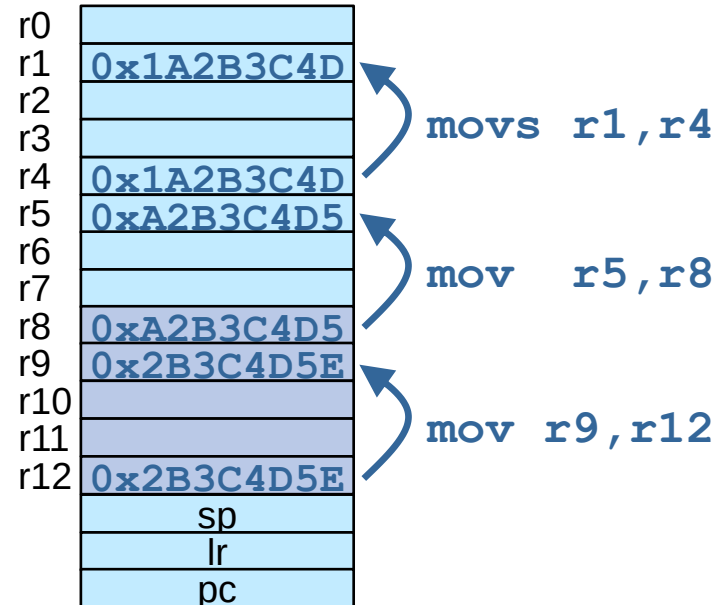
Transfer Types (continued)



¹⁾ Code region is typically read-only

Register to Register mov / movs (register)

- Copy register value to other register
- movs → low and high registers
- movs → only low registers
s = update of flags ³⁾



3-bit → low registers only

- 1) Instruction group "special data processing"
see table in lecture 2
- 2) Instruction group "shift by immediate, move register"
see table in lecture 2
- 3) 'S' stands for status register

Register to Register

Examples mov / movs

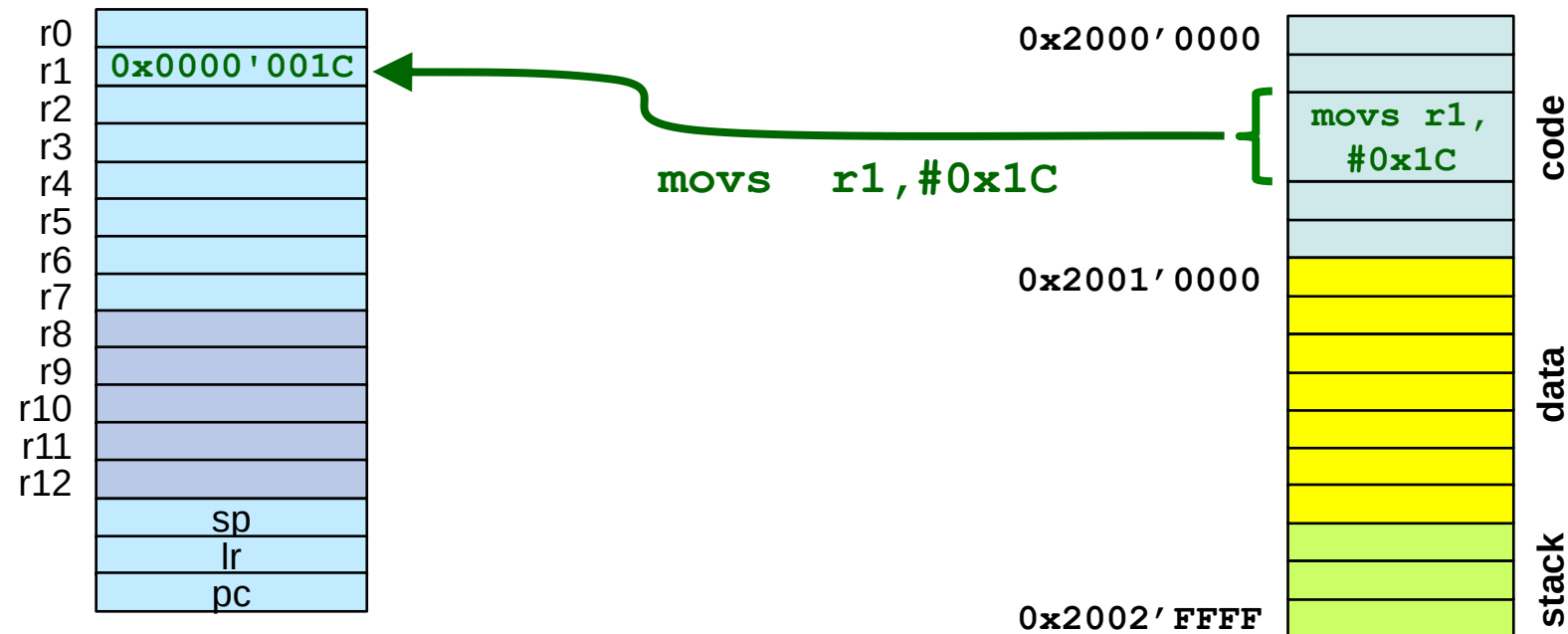
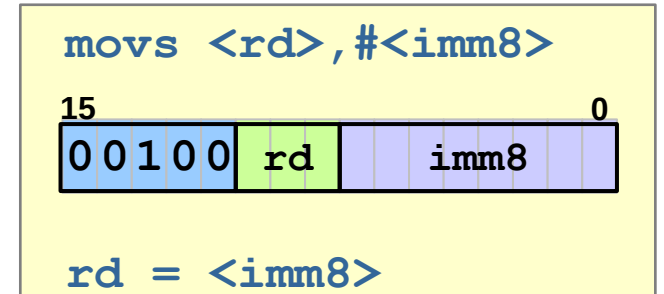
- Copy value of register Rm to Rd
 - **mov** <rd>, <Rr> high and low registers allowed
 - **movs** <rd>, <rm> restricted to low registers

instruction	comment
mov r1, r4	@ low reg to low reg
mov r1, r8	@ high reg to low reg
mov r8, r1	@ low reg to high reg
mov r8, r9	@ high reg to high reg
movs r1, r4	@ low reg to low reg
@ movs r1, r8	not possible: high reg
@ movs r8, r1	
@ movs r8, r9	

Loading Literals

MOVS (immediate data)

- Copy immediate 8-bit value (literal) to register (only low registers)
- 8-bit literal is part of opcode (imm8)
- Register-bits 31 to 8 set to 0



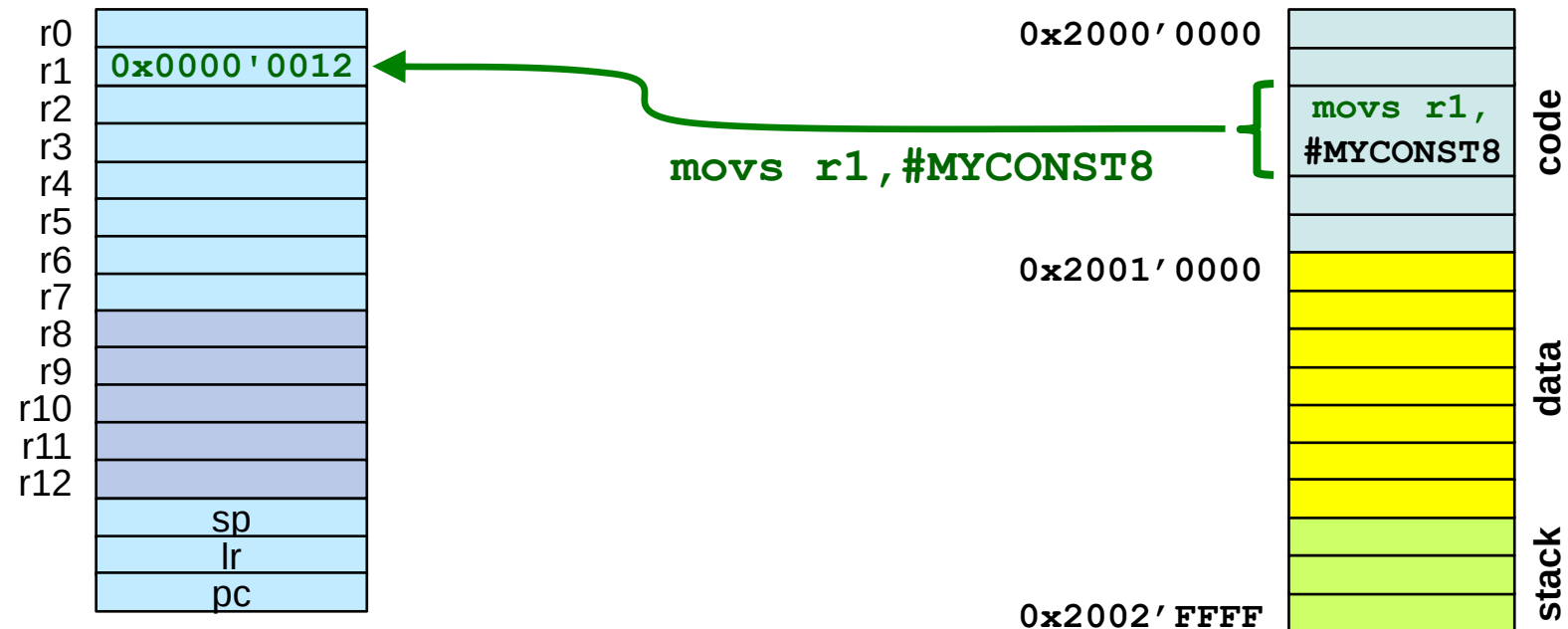
Loading Literals

EQU - Assembler Directive

- Symbolic definition of literals and constants
- Comparable to **#define** in C

Example:
movs with symbolic definition of literal

```
.equ MY_CONST8, 0xCD  
movs r1, #MY_CONST8
```



Loading Literals

Example MOVS (immediate data)

- Immediate 8-bit → 0 to 255d

```
.equ MY_CONST8, 0xCD           @ assembler directive
                                @ does not generate opcode

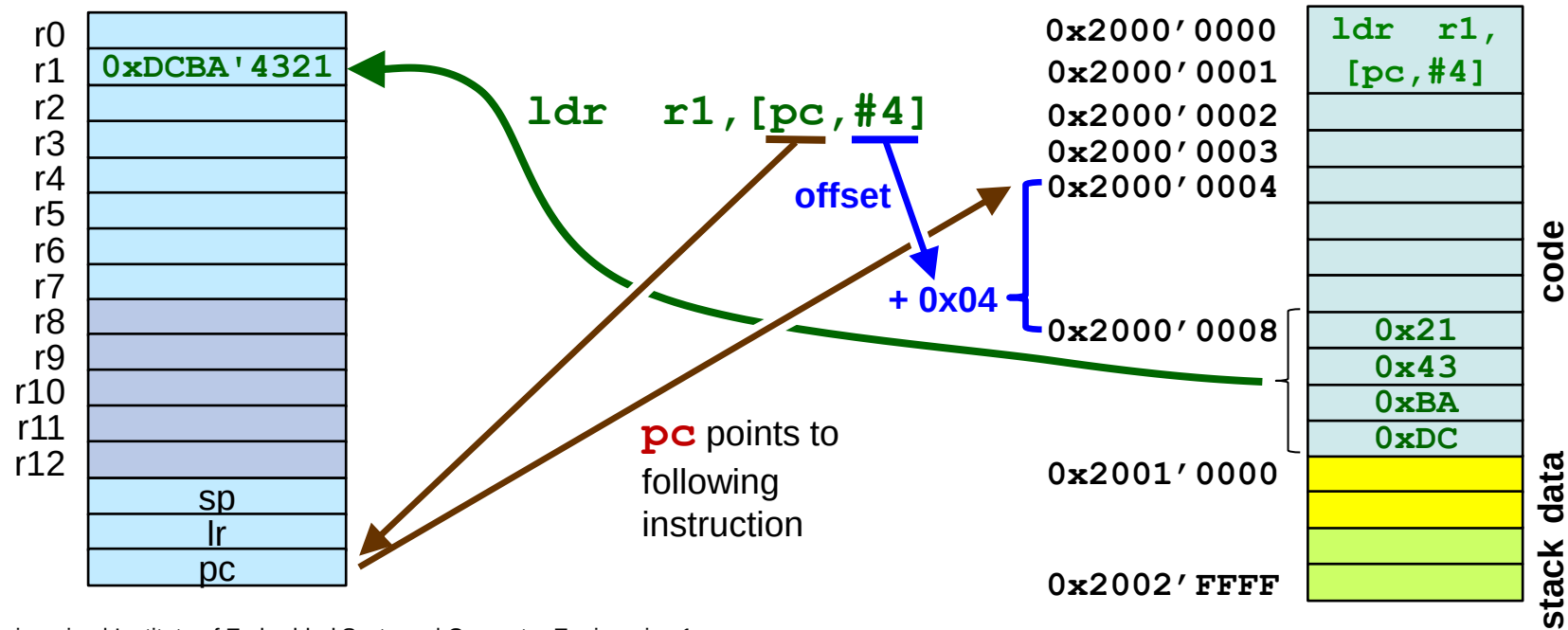
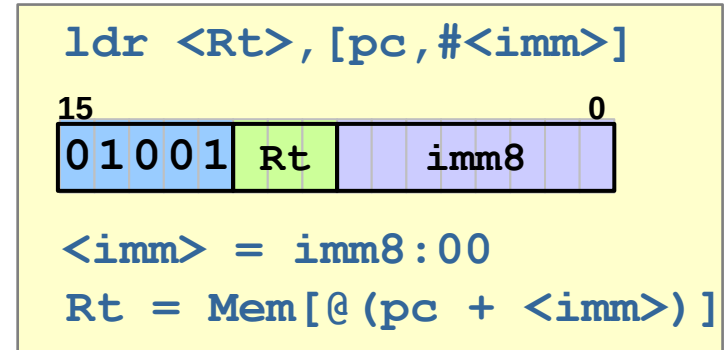
movs    r1, #0xFF              @ immediate hex to low reg
movs    r4, #12                 @ immediate dec to low reg
movs    r7, #MY_CONST8         @ literal with symbolic name

@ movs    r8, #MY_CONST8       high reg not possible
@ movs    r1, #0x100           immediate out of range
```

Loading Literals

Load - ldr (literal)

- Indirect access relative to pc ¹
- pc offset <imm>
- If pc not word-aligned
 - align on next upper word-address



¹ offsets are positive multiples of 4 in the range of 0 to 0x3FC, i.e. 1020d

Loading Literals

Example LDR (literal)

- Offset in bytes vs. imm8 in words $\rightarrow$ $\langle \text{imm} \rangle = \text{imm8}:00$ ¹⁾
 - Line14: assembler converts $\langle \text{imm} \rangle$ **0x08** to **imm8 = 0x02**
- Literals on lines 20 and 24
- pc points to following instruction
 - if not word-aligned $\rightarrow$ align on next upper word-address

00000014 <ldr_lit>:

```
14: 4902      ldr    r1, [pc, #0x08]    @ hex offset
16: 4a03      ldr    r2, [pc, #12]      @ dec offset
18: 4b01      ldr    r3, [pc, #4]
1a: 4c01      ldr    r4, [pc, #4]
1c: 4d00      ldr    r5, [pc, #0]
1e: e003      b      0x2c <ldr_lit2>
```

Pseudo instruction:
Assembler converts to
`ldr r3, [pc, #0x04]`

00000020 <myLit>:

```
20: 12345678    .word    0x12345678
24: 9abcdef0    .word    0x9abcdef0
```

¹⁾ offset = imm8 << 2 i.e. imm8 * 4

Loading Literals

Examples LDR (literal)

- Which values are being loaded into registers r1 to r5?

same code as previous slide

```
00000014 <ldr_lit>:
 14:  4902      ldr     r1, [pc, #0x08]  @ hex offset
 16:  4a03      ldr     r2, [pc, #12]    @ dec offset
 18:  4b01      ldr     r3, [pc, #4]
1a:  4c01      ldr     r4, [pc, #4]
1c:  4d00      ldr     r5, [pc, #0]
1e:  e003      b       0x2c <ldr_lit2>

00000020 <myLit>:
 20:  12345678  .word   0x12345678
 24:  9abcdef0  .word   0x9abcdef0
```

Use a symbol: Let assembler do the calculation

Branch prevents interpretation of literals as instructions

1) Word alignment of pc causes the same offset in lines 3 and 4, although they are accessing the same literal

Line 14:	pc = 0x00000016 --> word_aligned 0x00000018 literal from 0x00000018 + 0x08 = 0x00000020 r1 = 0x12345678	Load
Line 16	pc = 0x00000018 --> word_aligned 0x00000018 Load literal from 0x00000018 + 12d = 0x00000024 r2 = 0x9ABCDEF0	

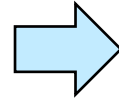
Loading Literals Pseudo Instruction

- Assembler
 - creates 'literal pool' at convenient code location
 - allocates and initializes memory in 'literal pool' → DCD
 - uses LDR (literal) instruction and calculates offset ¹

```
ldr    rd,=literal
```

assembly code as written
by human programmer

```
ldr    r1,=0x20000012
```



code generated
by assembler (tool)

```
ldr    r1, [pc, #68]
```

...

```
.word ...
```

```
.word 0x20000012
```

```
.word ...
```

'Literal Pool' at end of code block

'literal pool' → group of literals

¹ on M3/M4 the assembler may use a MOV instruction with an immediate value instead

Loading Literals

Pseudo Instruction Examples

```
.equ CONST_A = 0x000000AA
.equ CONST_B = 0xBBCCDDEE

ldr_lit3:
    ldr r7, =0x12
    ldr r7, =CONST_A
    ldr r7, =CONST_B
    ldr r5, mylita
    ldr r5, =mylita
    b    wherever

mylita:
    .word 0xFF001122

    .ltorg
```

load 0xFF001122 from
address 0x00000048

load address of mylita from
address 0x00000058

Generate literal pool here

Warning: difference between
ldr r5,mylita and **ldr r5,=mylita**

Loading Literals

Pseudo Instruction Examples

```
.equ CONST_A = 0x000000aa
.equ CONST_B = 0xbbccdee
ldr_lit3:
    ldr r7, =0x12
    ldr r7, =CONST_A
    ldr r7, =CONST_B
    ldr r5, mylita
    ldr r5, =mylita
    b    wherever

mylita:
    .word 0xFF001122

.ltorg
```

load 0xFF001122 from address 0x00000048

load address of mylita from address 0x00000058

Generate literal pool here

Literal Pool

```
0000003c <ldr_lit3>:
    3c:  4f03      ldr r7, [pc, #12]  @ (4c <mylita+0x4>)
    3e:  4f04      ldr r7, [pc, #16]  @ (50 <mylita+0x8>)
    40:  4f04      ldr r7, [pc, #16]  @ (54 <mylita+0xc>)
    42:  4d01      ldr r5, [pc, #4]   @ (48 <mylita>)
    44:  4d04      ldr r5, [pc, #16]  @ (58 <mylita+0x10>)
    46:  e009      b.n    5c <wherever>

00000048 <mylita>:
    48:  ff001122 .word 0xff001122

    4c:  00000012 .word 0x00000012
    50:  000000aa .word 0x000000aa
    54:  bbccddee .word 0xbbccddee
    58:  00000048 .word 0x00000048
```

CONST_A and CONST_B

space where the address of mylita will be stored after linking

Loading Literals

Pseudo Instruction Examples

```
ldr r5,mylita
```

→ `ldr r5,[pc,#...]`

The **value** `0xFF001122` at label `mylita` is loaded into `r5`

```
ldr r5,=0x20003000
```

→ `ldr r5,[pc,#...]`

space is allocated in literal pool. The **value** `0x20003000` is stored into this location and loaded into `r5` from there.

```
ldr r5,=CONST_A
```

→ `ldr r5,[pc,#...]`

space is allocated in literal pool. The **value** `0x000000AA`, defined through `=`, is stored into this location and loaded into `r5` from there.

```
ldr r5,=mylita
```

→ `ldr r5,[pc,#...]`

space is allocated in literal pool. The address of `mylita` is stored into this location and loaded into `r5` from there.

`=` sign tells the compiler to allocate and initialize space in literal pool. → Programmer does not have to type DCD lines.

```
.equ CONST_A, 0x000000AA  
  
.section .text.myCode, "ax", %progbits
```

```
mylita    b    wherever  
          .word 0xFF001122
```

Literal pool will be created here

Loading Literals

C Example

C-Code

```
static uint32_t g;

void lit_example(void) {
    uint32_t a;
    uint32_t b;
    const uint32_t c = 0x04;
    uint32_t *p;

    a = 0x05;
    b = 0xABCD1234; 1)
    p = &g;
    ...
}
```

Loading Literals

C Example

```
4904 → ldr r1,[pc,#0x10]
4a04 → ldr r2,[pc,#0x10]
```

C-Code

```
static uint32_t g;

void lit_example(void) {
    uint32_t a;
    uint32_t b;
    const uint32_t c = 0x04;
    uint32_t *p;

    a = 0x05;
    b = 0xABCDEF12; 1)
    p = &g;
    ...
}
```

Compiler assigns variables to registers	a → r0	p → r2
	b → r1	c → r3

Assembly

```
.section .text.myCode, "ax", %progbits
...
000002 2304      movs    r3,#4
000004 2005      movs    r0,#5
000006 4904      ldr     r1,lit1
000008 4a04      ldr     r2,adr_g 2)
...
000018      lit1:   .word 0xabcdef12
00001C      adr_g:  .word g 3)

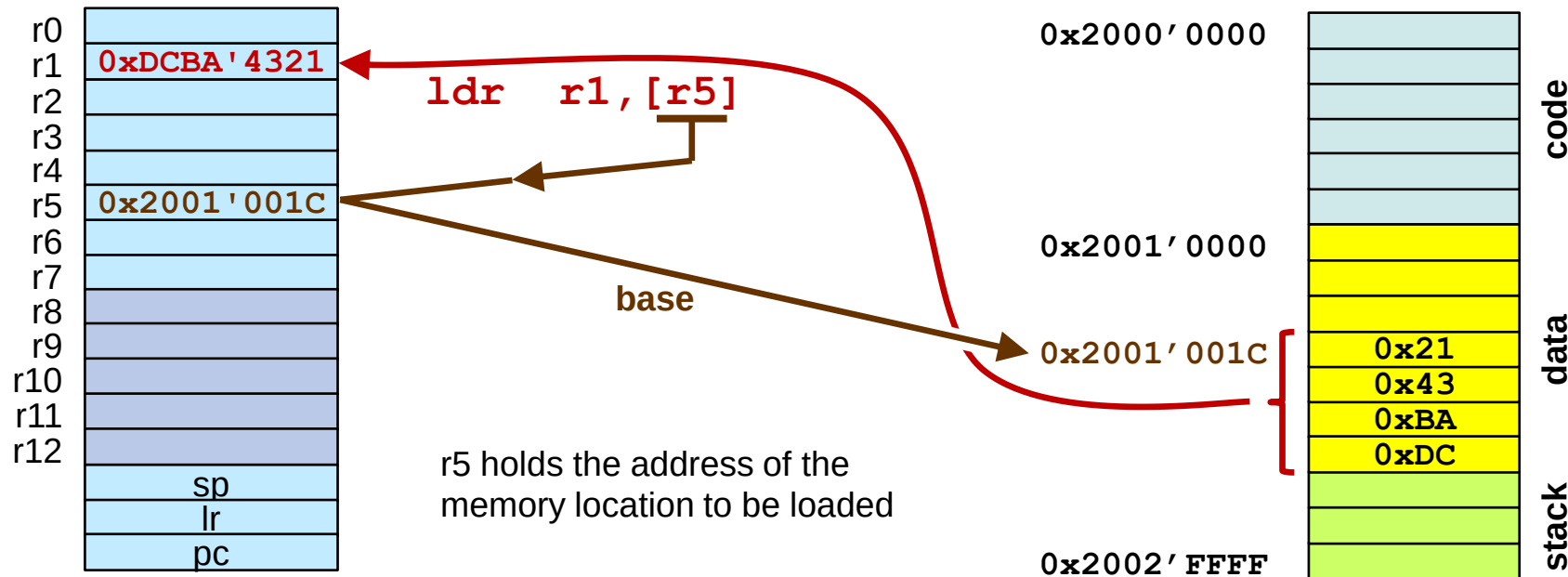
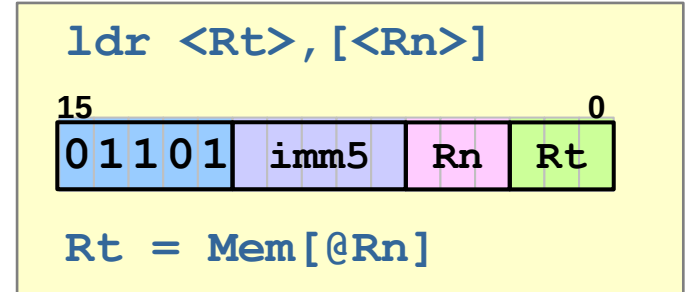
.section .progData, "aw", %progbits
g:      .word 0x00000000
```

- 1) Compiler translates `b = 0xABCDEF12;` to `LDR r1,lit1`
Assembler then translates `LDR r1,lit1` to `LDR r1,[pc,#0x10]`
- 2) loads the address of `g`
- 3) stores the address of `g` (not the content of `g`)
Compiler does not use the pseudo-instruction `LDR r2,=g`, which would mean the same.

Loading Data

Ldr (immediate offset) – imm5 = 0 ¹

- Indirect addressing → [...]
- Rn → memory address
- Only low registers

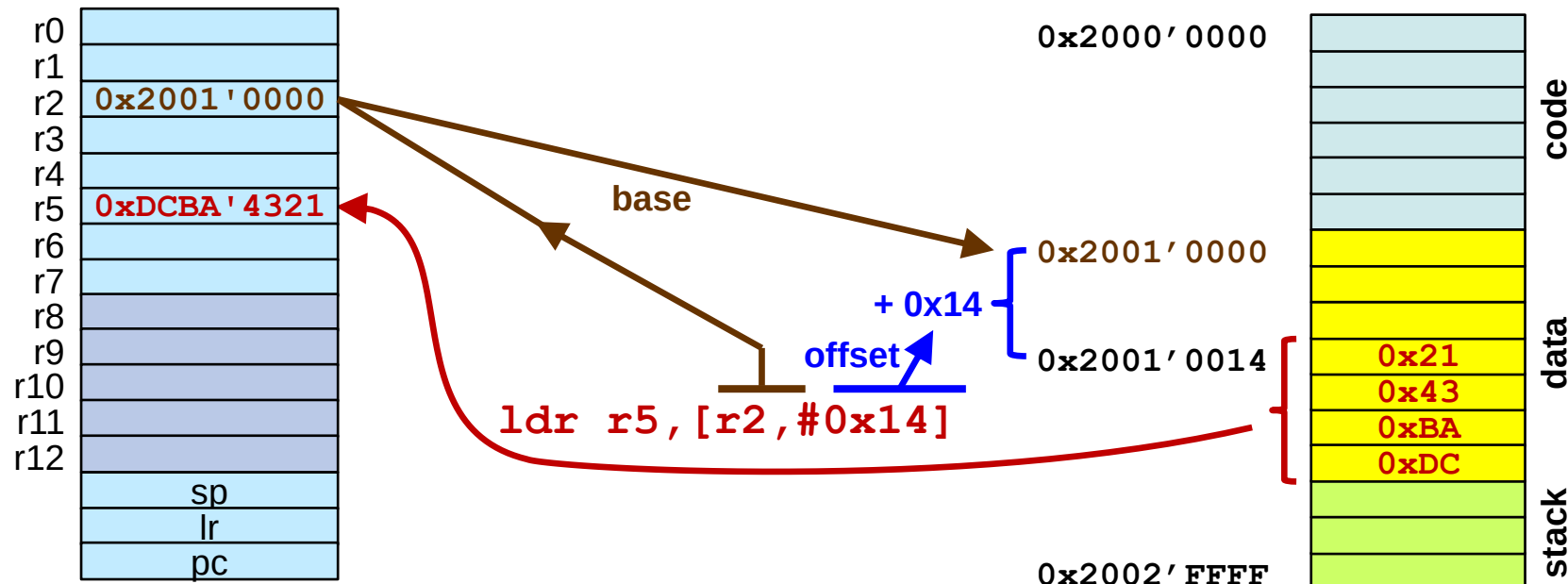
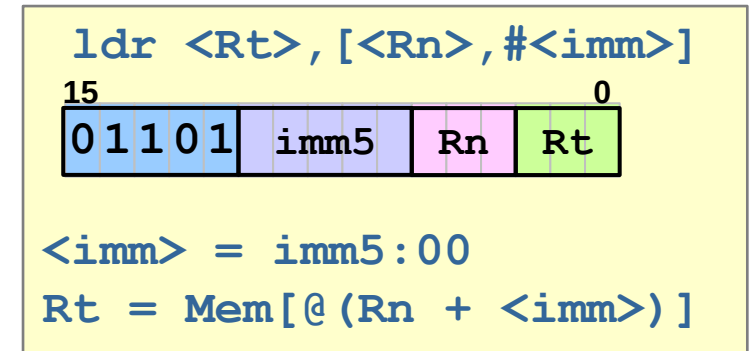


¹ imm5 = 0 is a special case of the general LDR (immediate offset) on the next slide.

Loading Data

ldr (immediate offset) – general

- Indirect addressing
 - Immediate offset <imm>
 - Offset range 0 - 124d (0x7C) ¹
- Only low registers

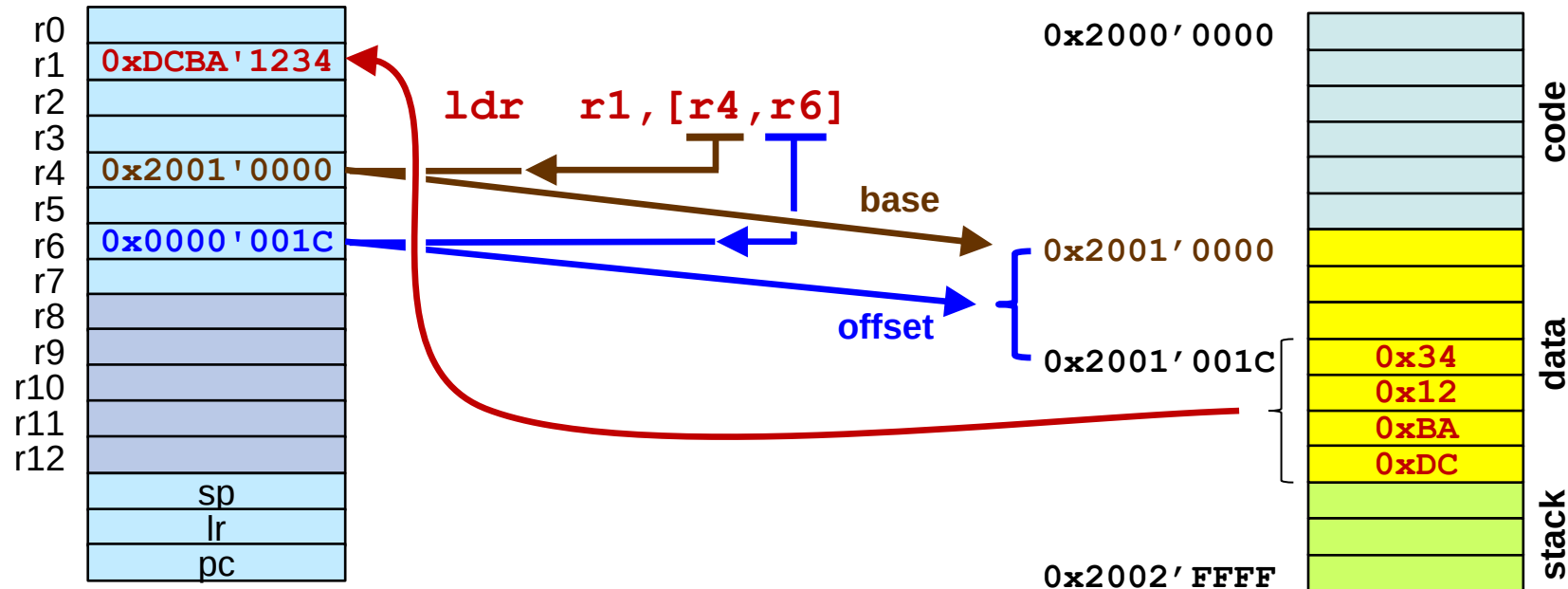
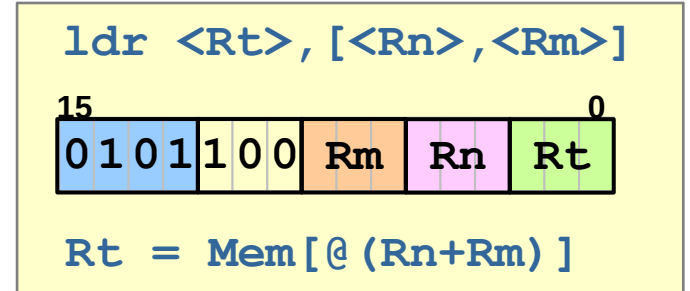


¹ positive values, multiples of 4 only

Loading Data

ldr (register offset)

- Indirect addressing with offset register
 - offset = unsigned
- Only low registers



Loading Data Examples

- Content of r4, r5, r6 after execution?

```
.section .progData, "aw", %progbits
my_array:
.word 0x11223344
.word 0x55667788
.word 0x99AABBCC
```

```
.section .text.myCode, "ax", %progbits
. . .

@ load base and offset registers
ldr    r1,=my_array    @ load address of array
ldr    r3,=0x08
```

```
@ indirect addressing
ldr    r4,[r1]          ; base r1
ldr    r5,[r1,#0x04]    ; base r1, immediate offset
ldr    r6,[r1,r3]       ; base r1, offset r3
```

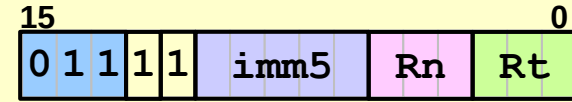
Not content of my_array,
but address of my_array

Loading Data

Byte / Halfword

- `ldrb` (register/immediate offset)
 - Load Register Byte
 - Register bits 31 to 8 set to zero
- `ldrh` (register/immediate offset)
 - Load Register Half-word
 - Register bits 31 to 16 set to zero

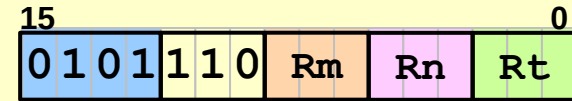
```
ldrb <Rt>, [<Rn>, #<imm>]
```



$\langle imm \rangle = imm5$

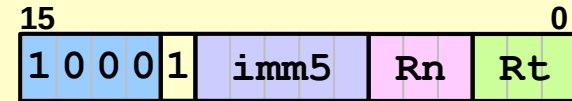
$Rt = \text{Byte}[@(Rn + \langle imm \rangle)]$

```
ldrb <Rt>, [<Rn>, <Rm>]
```



$Rt = \text{Byte}[@(Rn + Rm)]$

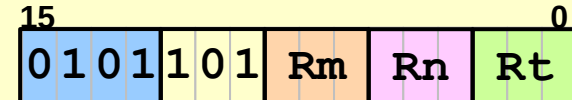
```
ldrh <Rt>, [<Rn>, #<imm>]
```



$\langle imm \rangle = imm5:0$

$Rt = \text{Hw}[@(Rn + imm5)]$

```
ldrh <Rt>, [<Rn>, <Rm>]
```

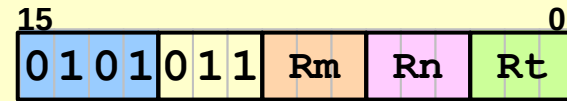


$Rt = \text{Hw}[@(Rn + Rm)]$

Loading Data Signed

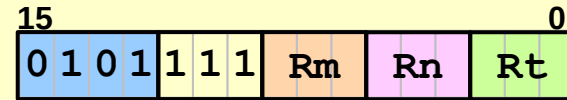
- ldrsb
 - Load Register Signed Byte
 - Sign extend
 - Register bits 31 to 8 set or reset depending on bit 7
- ldrsh
 - Load Register Signed Half-word
 - Sign extend
 - Register bits 31 to 16 set or reset depending on bit 15
- Sign Extension
 - See slides on casting

```
ldrsb <Rt>, [<Rn>, <Rm>]
```



```
Rt = sign_extend(Byte[@(Rn+Rm)])
```

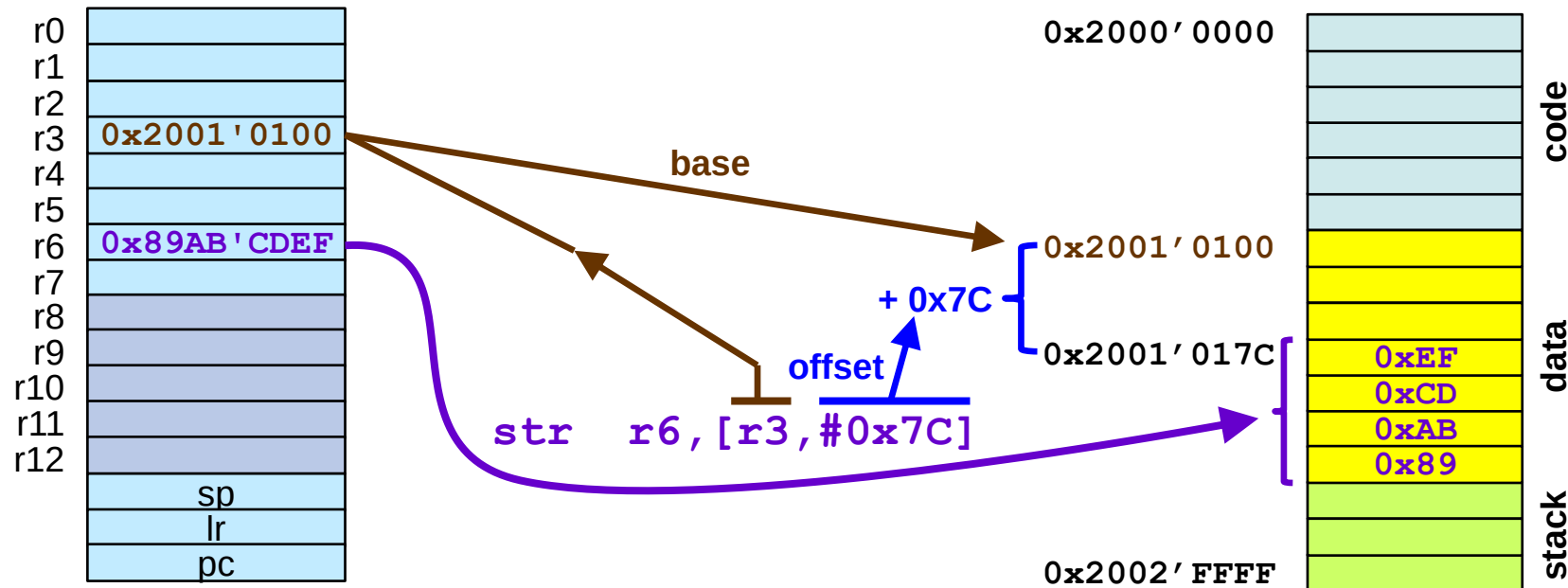
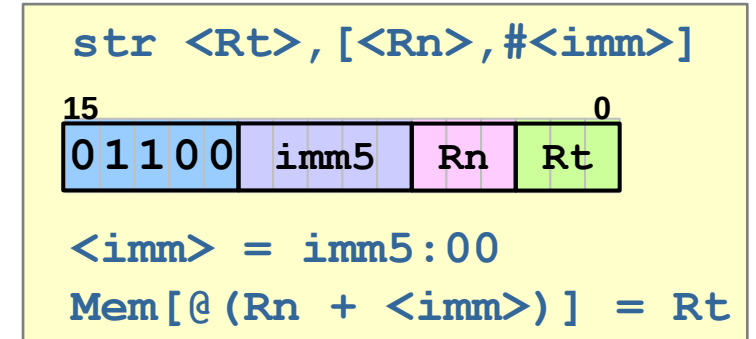
```
ldrsh <Rt>, [<Rn>, <Rm>]
```



```
Rt = sign_extend(Hw[@(Rn+Rm)])
```

Storing Data str (immediate offset)

- Indirect addressing with immediate offset
 - Offset range 0 - 124d (0x7C)¹
- Only low registers

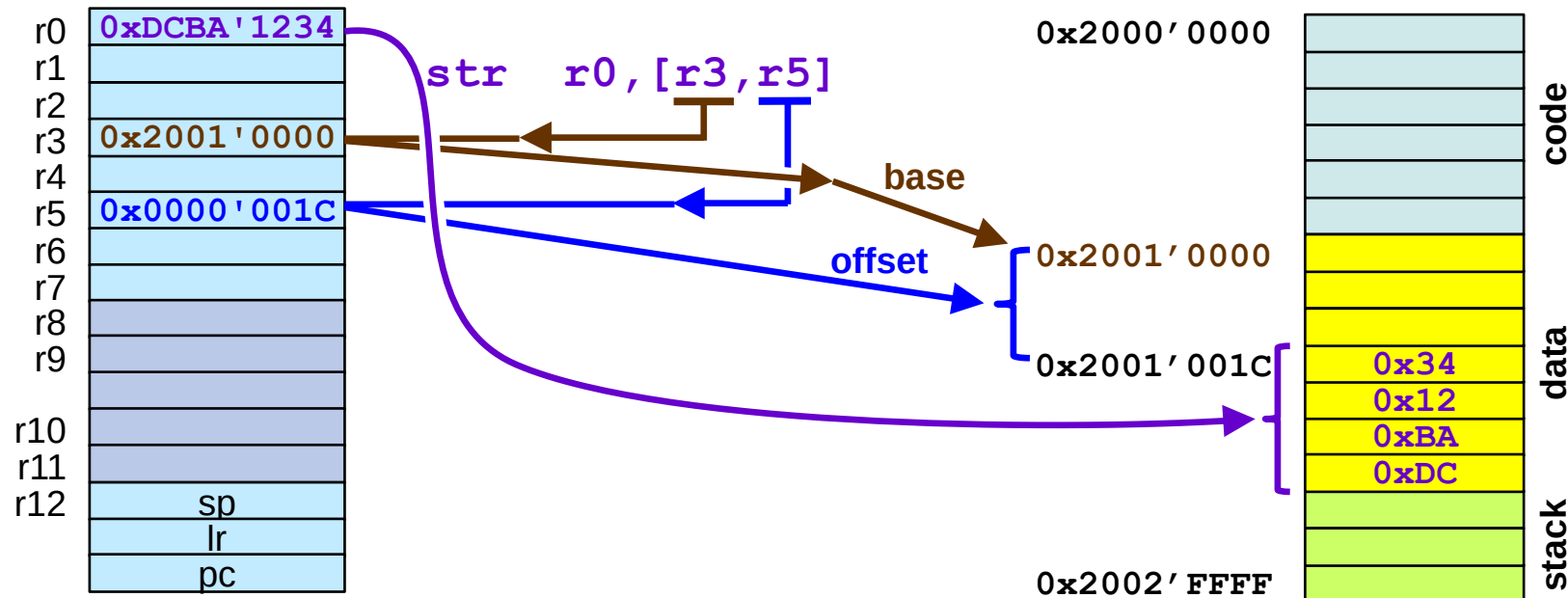
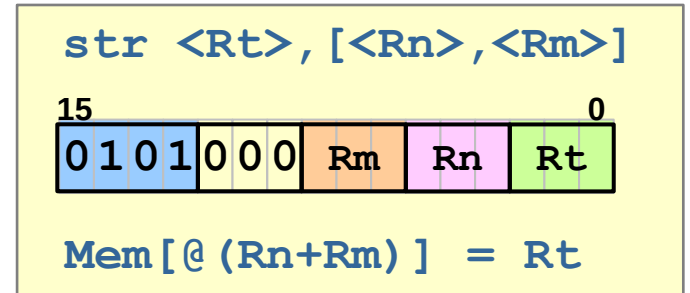


¹ positive values, multiples of 4 only

Storing Data

str (register offset)

- Indirect addressing with offset register
 - Offset register $\rightarrow$ index
- Only low registers



Storing Data

Example Source Code

```
.section .progData, "aw", %progbits  
data_array:  
.space 256
```

```
ldr    r1, =CONST_C  
ldr    r2, =CONST_D  
ldr    r3, =CONST_E  
ldr    r7, =data_array  
ldr    r6, =0x08  
str    r1, [r7]  
str    r2, [r7, #0x04]  
str    r3, [r7, r6]  
b      ldm_ex
```

```
CONST_C    .word    0xCCCCCCCC  
CONST_D    .word    0xDDDDDDDD  
CONST_E    .word    0xEEEEEEEE
```

store CONST_C in memory at
address of data_array

store CONST_D in memory at
address of data_array + 0x04

store CONST_E in memory at
address of data_array + 0x08

Storing Data

Example Listfile

```
.section .progData, "aw", %progbits
data_array:
.space 256
```

```
CONST_C .word 0xCCCCCCCC
CONST_D .word 0xDDDDDDDD
CONST_E .word 0xEEEEEEEE
```

```
a0: 4904      ldr    r1, [pc, #16] @ (b4 <str_ex+0x14>)
a2: 4a05      ldr    r2, [pc, #20] @ (b8 <str_ex+0x18>)
a4: 4b05      ldr    r3, [pc, #20] @ (bc <str_ex+0x1c>)
a6: 4f06      ldr    r7, [pc, #24] @ (c0 <str_ex+0x20>)
a8: 4e06      ldr    r6, [pc, #24] @ (c4 <str_ex+0x24>)
aa: 6039      str    r1, [r7, #0]
ac: 607a      str    r2, [r7, #4]
ae: 51bb      str    r3, [r7, r6]
b0: e00a      b.n    c8 <ldm_ex>
b2: 46c0      nop
b4: CCCCCCCC .word 0xCCCCCCCC
b8: DDDDDDDD .word 0xDDDDDDDD
bc: EEEEEEEE .word 0xEEEEEEEE
c0: 00000000 .word 0x00000000
c4: 00000008 .word 0x00000008
```

store CONST_C in memory at
address of data_array

store CONST_D in memory at
address of data_array + 0x04

store CONST_E in memory at
address of data_array + 0x08

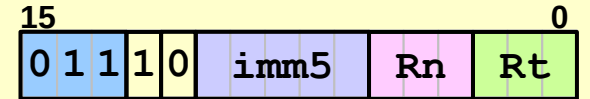
storage space in literal pool for address of data_array

Storing Data

Byte / Halfword

- strb (immediate/register offset)
 - Store Register Byte
 - Low 8 bits of register stored
- strh (immediate/register offset)
 - Store Register Half-word
 - Low 16 bits of register stored

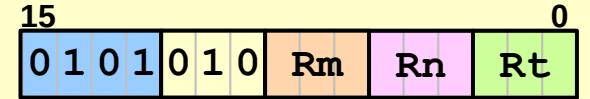
`strb <Rt>, [<Rn>, #<imm>]`



`<imm> = imm5`

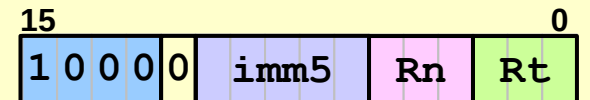
`Byte[@ (Rn+<imm>)] = Rt(7:0)`

`strb <Rt>, [<Rn>, <Rm>]`



`Byte[@ (Rn+Rm)] = Rt(7:0)`

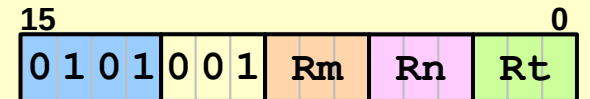
`strb <Rt>, [<Rn>, #<imm>]`



`<imm> = imm5:0`

`Hw[@ (Rn+<imm>)] = Rt(15:0)`

`strb <Rt>, [<Rn>, <Rm>]`



`Hw[@ (Rn+Rm)] = Rt(15:0)`

Summary Data Transfer

`movs <rd>,<rm>`

Register to register

`movs <rd>,#<imm8>`

Loading literals

8-bit literal

`ldr <rt>,[pc,#<imm>]`

32-bit literal, pc-relative

`ldr <rt>,[<rn>,#<imm>]`
also `ldrb` and `ldrh`

Loading data

Register indirect with immediate offset

`ldr <rt>,[<rn>,<rm>]`
also `ldrb`, `ldrh`, `ldrsb` and `ldrsh`

Register indirect with register offset

`str <rt>,[<rn>,#<imm>]`
also `strb` and `strh`

Storing data

Register indirect with immediate offset

`str <rt>,[<rn>,<rm>]`
also `strb` and `strh`

Register indirect with register offset

Loading/Storing Multiple Registers

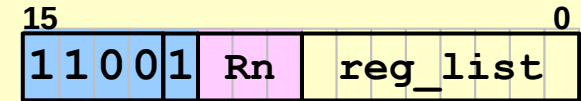
ldm ¹

For information only

- Load Multiple Registers
- Rn: Base address

```
ldm <Rn>!, <registers>
```

```
ldm <Rn>, <registers>
```



Registers in `reg_list` are loaded from memory starting at address in `Rn`

000000c8 <ldm_ex>:

```
8: 4a06 ldr r2, [pc, #24] @ (e4 <ldm_const+0x14>)
ca: cae6 ldm r2, {r1, r2, r5, r6, r7}
cc: e00c b.n e8 <ldm_ex2>
ce: 46c0 nop
```

000000d0 <ldm_const>:

```
d0: AAAAAAAAAA .word 0xAAAAAAAA
d4: BBBBBBBBBB .word 0xBBBBBBBB
d8: CCCCCCCC .word 0xCCCCCCCC
dc: DDDDDDDD .word 0xDDDDDDDD
e0: EEEEEEEE .word 0xEEEEEEEE
e4: 000000D0 .word 0x000000D0
```

r1 = 0xAAAA'AAAA
r2 = 0BBBBB'BBBB
r5 = 0xCCCC'CCCC
r6 = 0xDDDD'DDDD
r7 = 0xEEEE'EEEE

¹ `ldmia` (Load Multiple Increment After) and `ldfmd` (Load Multiple from Full Descending stacks) are aliases for `LDM`

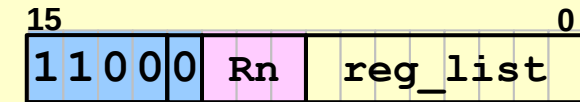
Loading/Storing Multiple Registers

stm ¹

For information only

- Store Multiple Registers
- Rn: Base address

`stm <Rn>!, <registers>`



Registers in `reg_list` are stored to memory starting at address in `Rn`

```
.section .progData, "aw", %progbits
data_array:
.space 256
```

```
000000f4 <stm_ex>:
```

```
f4: 4c02      ldr      r4, [pc, #8]    @ (100 <stm_ex+0xc>)
f6: c4e6      stmia   r4!, {r1, r2, r5, r6, r7}
f8: e004      b.n     104 <loop>
fa: 46c0      nop                      @ (mov r8, r8)
fc: 000000D0  .word    0x000000D0
100: 00000000  .word    0x00000000
```

r1 → @data_array
r2 → @data_array + 0x04
r5 → @data_array + 0x08
r6 → @data_array + 0x0C
r7 → @data_array + 0x10

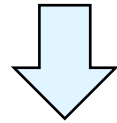
¹ STMEA (Store Multiple Increment After) and STMEA (Store Empty Ascending) are aliases for STM

The C Perspective

Array of Bytes

C-code

```
static uint8_t byte_array[] =  
    {0xAA, 0xBB, 0xCC, 0xDD,  
     0xEE, 0xFF};
```



assembly

```
byte_array:  
    .byte    0xAA, 0xBB, 0xCC, 0xDD  
    .byte    0xEE, 0xFF
```

address		index
0x2001'0000	0xAA	0
0x2001'0001	0xBB	1
0x2001'0002	0xCC	2
0x2001'0003	0xDD	3
0x2001'0004	0xEE	4
0x2001'0005	0xFF	5

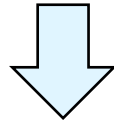
assuming that `byte_array` starts at 0x2001'0000

The C Perspective

Array of Half-Words

C-code

```
static uint16_t halfword_array[] =  
    {0x0011, 0x2233,  
     0x4455, 0x6677,  
     0x8899, 0xAABB};
```



assembly

```
halfword_array:  
    .hword    0x0011,0x2233  
    .hword    0x4455,0x6677  
    .hword    0x8899,0xAABB
```

address		index
0x2001'0000	0x11	0
	0x00	
0x2001'0002	0x33	1
	0x22	
0x2001'0004	0x55	2
	0x44	
0x2001'0006	0x77	3
	0x66	
	0x99	4
	0x88	
	0xBB	5
	0xAA	
0x2001'000F		

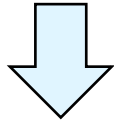
assuming that `halfword_array` starts at 0x2001'0000

The C Perspective

Array of Words

C-code

```
static uint32_t word_array[] =  
    {0xFFEEDDCC,  
     0xBBAA9988,  
     0x77665544,  
     0x33221100};
```



assembly

```
word_array:  
    .word    0xFFEEDDCC  
    .word    0xBBAA9988  
    .word    0x77665544  
    .word    0x33221100
```

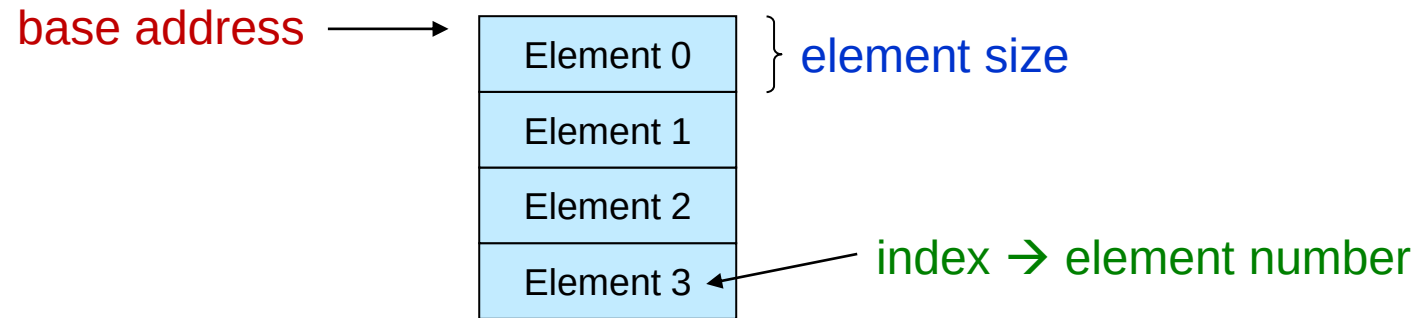
address		index
0x2001'0000	0xCC	0
	0xDD	
	0xEE	
	0xFF	
0x2001'0004	0x88	1
	0x99	
	0xAA	
	0xBB	
0x2001'0008	0x44	2
	0x55	
	0x66	
	0x77	
0x2001'000C	0x00	3
	0x11	
	0x22	
0x2001'000F	0x33	

assuming that `word_array` starts at 0x2001'0000

The C Perspective

Accessing Array Elements

- element address = **base address** + **element size** • **index**
- element sizes in bytes
 - word 4
 - half-word 2
 - byte 1



The C Perspective

Example: Array Access

C-Code

```
static uint8_t byte_array[] =
    {0xAA, 0xBB, 0xCC, 0xDD,
     0xEE, 0xFF};

void access_byte_array(void)
{
    ...
    byte_array[3] = 0x12;
    ...
}
```

Assembly

```
.section .progData, "aw", %progbits
byte_array:
    .byte 0xaa,0xbb
    .byte 0xcc,0xdd
    .byte 0xee,0xff

.section .text.myCode, "ax", %progbits
access_byte_array:
    ...
1  movs    r0,#0x12
2  ldr     r1,adr_b
3  strb    r0,[r1,#3]
    ...
adr_b:
    .word byte_array
```

- 1 Load value to be stored into r0
- 2 Load base address from label below ¹⁾
- 3 Store r0 to base address plus offset

¹⁾ Compiler picks this form and not `LDR r1,=byte_array`

The C Perspective

Array access (word)

C-Code

```
static uint32_t word_array[] =
    {0xFFEEDDCC,
     0xBBAA9988,
     0x77665544,
     0x33221100};

void access_word_array(void)
{
    ...
    word_array[3] = 0xAABBCCDD;
    ...
}
```

Assembly

```
.section .progData, "aw", %progbits
word_array:
    .word    0xffeeddcc
    .word    0xbbaa9988
    .word    0x77665544
    .word    0x33221100

.section .text.myCode, "ax", %progbits
access_word_array:
    ...
1  ldr      r0, lit_1
2  ldr      r1, adr_w
3  ldr      r0, [r1, #0xc]
    ...

lit_1:
    .word    0xaabbccdd
adr_w:
    .word    word_array
```

- 1 Load literal from label → r0
- 2 Load base address from label → r1
- 3 Store r0 to base address plus offset
*offset (0xc) = element size (4) * index (3)*

The C Perspective

Pointer and Address Operator

C-Code

```
void pointer_example(void)
{
    static uint32_t x;
    static uint32_t *xp;

    xp = &x;
    *xp = 0x0C;
}
```

Assembly

```
.section .progData, ...
x:  .word    0x00000000
xp: .word    0x00000000

.section .text.myCode, ...
pointer_example:
    ...
1 ldr      r0, adr_x
2 ldr      r1, adr_xp
3 str      r0, [r1, #0]
4 movs     r0, #0xc
5 ldr      r1, [r1, #0]
6 str      r0, [r1, #0]
    ...

adr_x:  .word    x
adr_xp: .word    xp
```

- 1 Load address of x → r0
- 2 Load address of xp → r1
- 3 Store r0 (i.e. address of x) in xp variable
(indirect memory access through r1)
- 4 Load immediate value 0x0C → r0
- 5 Load content of xp → r1
i.e. address of x is now in r1
- 6 Store r0 at address given by r1

The C Perspective: Pointer and Memory Mapped I/O

- Write to LEDs on CT Board

C-Code

```
void main(void)
{
    volatile uint32_t *p;

    p = (volatile uint32_t *)
        0x60000100;
    *p = 0x1A2B3C4D;
}
```

Assembly

```
.text
ldr    r0,led_adr
ldr    r1,led_val
str    r1,[r0, #0]

led_val:
.word 0x1A2B3C4D
led_adr:
.word 0x60000100
```

The local variable p is kept in register r0, not in memory

Conclusion

- **Data transfers**

- Register to Register `MOV/MOVS (register)`
- Loading Literals `movs (immediate data)`
`ldr (pc-relative/literal-pool)`
- Loading Data `ldr (immediate/register offset)`
- Storing Data `str (immediate/register offset)`

- **Addressing Modes**

- pc Relative `[pc, #0x12]`
- Indirect Addressing `[r1], [r2, #0x12], [r5, r6]`

- **Arrays**

- Element address = `base address + element size • index`
- Accessed with data transfer instructions

- **Volatile**

- Use for accessing memory mapped items